

PORTFOLIO · AWS INFRASTRUCTURE

HSA 프로젝트

AWS 서버 인프라 구축 기록

한양대학교 ERICA 클라우드캡스톤디자인 — HSA(Hanyang Support Agent)
AI 기반 고객문의 자동분류·응답 에이전트의 백엔드/AI 서비스를 위한
AWS 인프라를 설계하고 배포하며 기록한 문서입니다.

ECS Fargate

VPC 설계

RDS PostgreSQL

Cloud Map

GitHub Actions CI/CD

IAM / Secrets Manager

작성자 · 최유탍 | 역할 · AI(RAG 워크플로우 구현) · AWS 인프라 설계 및 배포

리전 · us-east-1 (N. Virginia) | 수업/프로젝트 · 한양대학교 ERICA 클라우드캡스톤디자인 (HSA)

최종 업데이트 · 2026-06-23 | 민감 정보(비밀번호, 계정 일부)는 마스킹 처리했습니다.

목차

1. 프로젝트 개요 및 담당 역할
2. 리전 및 가용영역 구성
3. 전체 아키텍처 다이어그램
4. 다이어그램 구성 요소 설명
5. 네트워크 아키텍처 상세 (VPC · 서브넷 · 게이트웨이)
6. 트래픽 플로우 상세
7. ECS 컨테이너 서비스 상세
8. 서비스 디스커버리 (AWS Cloud Map)
9. 보안 그룹 설계
10. 컨테이너 레지스트리 (ECR)
11. 로드 밸런서 (ALB)
12. 데이터베이스 (RDS) 상세
13. IAM 및 보안 (KMS 포함)
14. 배포 파이프라인 (CI/CD)
15. OpenAI API 연동 구성
16. DNS 및 도메인 관리 (Route 53)
17. 모니터링 및 로깅
18. 운영 고려사항
19. 트러블슈팅 및 기술적 의사결정
20. 배운 점 및 향후 개선 방향
- 21. 프로젝트 결과 화면 (Frontend 실행 화면)**

1. 프로젝트 개요 및 담당 역할

프로젝트 소개

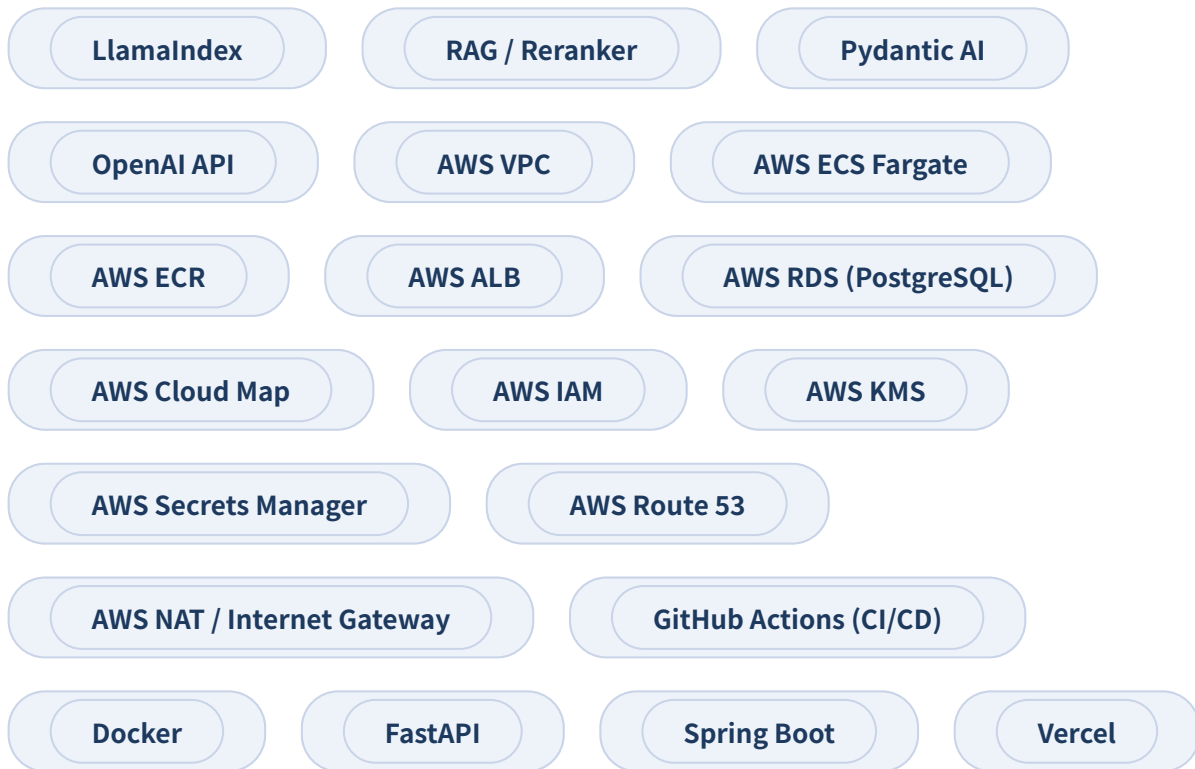
HSA(Hanyang Support Agent)는 1인·신생 쇼핑몰 운영자를 대상으로, 여러 채널(카카오톡·인스타그램·이메일)로 들어오는 고객 문의를 자동으로 분류하고, 단순 조회성 문의는 DB 조회 기반으로 자동응답하며, 정책·상품 판단이 필요한 문의는 RAG(Retrieval-Augmented Generation) 기반 답변 초안을 생성해 관리자의 응대를 돕는 AI Agent 시스템입니다. 시스템은 Frontend ↔ Backend ↔ AI 3개 서비스로 구성됩니다. 저는 AI 파트에서 정책 문서 검색·답변 생성을 담당하는 RAG 워크플로우 구현과, 이를 포함한 Backend(Spring Boot)·AI(FastAPI) 서비스를 운영하기 위한 AWS 인프라 설계 및 배포를 함께 맡았습니다. 이 문서는 그중 AWS 인프라 구축 과정에 집중해서 정리한 기록입니다.

담당 역할

AI · RAG 구현	정책 문서(배송/교환·환불/상품/응대 톤) Markdown화 및 LlamaIndex 벡터 인덱싱, 유사도 검색 → LLM reranker → 답변 초안 생성 파이프라인 구현, relevance threshold · prompt injection 방어 적용
인프라 설계	VPC/서브넷/보안그룹 등 네트워크 구조 설계, Public/Private 서브넷 분리, 최소 권한 보안 그룹 정책 수립
컴퓨트 · 배포	ECS Fargate 기반 컨테이너 서비스(Backend, AI) 구성, ECR 이미지 관리, Cloud Map 서비스 디스커버리 연결
데이터 · 보안	RDS(PostgreSQL) 구성 및 접근 제어, Secrets Manager를 통한 OpenAI API 키 관리, KMS 암호화 적용
CI/CD	GitHub Actions 기반 빌드·배포 파이프라인 구성 (OIDC 인증, ECS 롤링 배포, Circuit Breaker 롤백)
네트워크 · DNS	ALB/NAT Gateway/Internet Gateway 구성, Route 53 도메인 라우팅, Vercel(프론트엔드)과의 연동 구조 설계
문서화 · 검증	인프라 변경 사항을 문서로 추적하고, 기존 설계 문서와 실제 설정값을 대조해 보안 설정 오류를 점검·수정

AI/RAG 구현의 세부 내용(프롬프트 설계, 평가 지표 등)은 별도 문서에서 다루며, 이 문서는 AWS 인프라 구축 부분을 중심으로 다룹니다.

기술 스택



시스템 연결 방향

```
Frontend (Vercel) <-> Backend (Spring Boot, ECS Fargate) <-> AI (FastAPI, ECS Fargate)
                        |
                        v
                    RDS (PostgreSQL)
```

Backend는 문의 저장, 주문/배송 DB 조회, AI 호출, 고객 응답 발송, 로그 저장을 담당하고, AI는 Backend가 전달한 문의를 바탕으로 분류·자동응답 판단·RAG 답변 초안을 생성해 돌려주는 보조 서비스입니다. AI는 프론트엔드나 고객 채널과 직접 통신하지 않습니다.

문서 작성 기준

인프라 구축 과정에서 작성한 두 버전의 설계 문서(초기 스냅샷과 개정본)를 대조해 최신 상태로 통합했습니다. 두 문서가 충돌하는 부분은 더 최근에 개정된 내용을 기준으로 채택했고, 어떤 부분을 왜 바꿨는지는 19장 "트러블슈팅 및 기술적 의사결정"에 정리했습니다.

2. 리전 및 가용영역 구성

항목	내용
주 리전	us-east-1 (US East – N. Virginia)
가용 영역(AZ)	us-east-1a, us-east-1b
계정 ID	325899476013
선정 이유	낮은 지연시간, AWS 서비스 가용성, OpenAI API 접근성

3. 전체 아키텍처 다이어그램

AWS 공식 아이콘 양식으로 제작한 아키텍처 다이어그램입니다. 사용자 트래픽(실선), 내부/아웃바운드 트래픽(실선), 미래에 구현할 목표 구조(점선)를 구분해 표시했습니다.

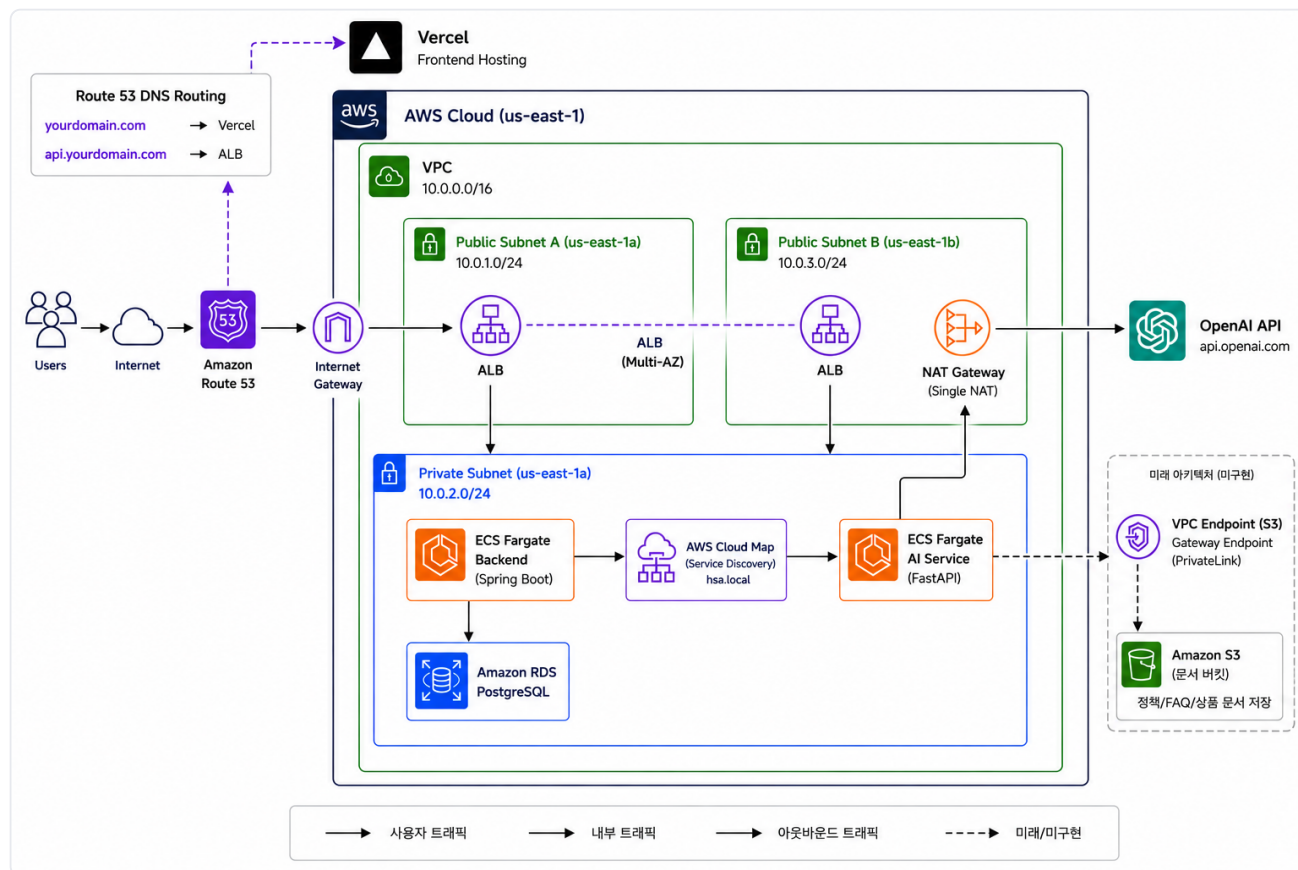


그림 1. HSA 배포 아키텍처 (AWS 공식 아이콘 다이어그램)

4. 다이어그램 구성 요소 설명

다이어그램에는 ALB가 두 퍼블릭 서브넷(us-east-1a, us-east-1b)에 걸쳐 표시되어 있는데, 이는 ALB 한 대가 다중 AZ로 구성되어 있다는 의미입니다 (서로 다른 두 대가 아닙니다). NAT Gateway는 단일 구성(Single NAT)으로, 해당 AZ에 장애가 발생하면 AI 서비스의 OpenAI API 호출(아웃바운드)이 영향을 받을 수 있는 구조입니다 (18장 참고).

남은 확인 사항 · NAT Gateway 소속 AZ

다이어그램에는 NAT Gateway가 Public Subnet B(us-east-1b, 10.0.3.0/24)에 위치한 것으로 그려져 있습니다. 그런데 5장에 기록해 둔 리소스 정보에는 NAT Gateway의 프라이빗 IP가 `10.0.1.207` 로, 이는 Public Subnet A(us-east-1a, 10.0.1.0/24) 대역에 속합니다. 둘 중 하나가 최신 상태를 반영하지 못한 것으로 보이며, 콘솔에서 실제 소속 서브넷을 다시 확인해 일치시킬 계획입니다.

구성 요소	역할
Route 53	프론트엔드(Vercel) · API(ALB) 도메인 라우팅
Vercel	프론트엔드 정적/SSR 호스팅, 자동 SSL — 확정 구조
Internet Gateway	퍼블릭 서브넷의 인터넷 연결 통로 (인바운드 ALB 진입 + 아웃바운드 NAT 경유 트래픽 모두 통과)
Application Load Balancer	API 요청을 ECS Backend로 라우팅. SG는 80/443 인바운드만 허용 (퍼블릭 서브넷)
NAT Gateway	프라이빗 서브넷에서 OpenAI API 호출 시 사용하는 아웃바운드 경로 (퍼블릭 서브넷에 위치)
ECS Fargate – BE (Spring Boot)	Backend API 서버. 인바운드는 ALB SG에서만 허용
ECS Fargate – AI (FastAPI)	문의 분류 · 자동응답 판단 · RAG 초안 생성. 인바운드는 BE SG에서만 허용
Amazon RDS (PostgreSQL)	주문/배송 등 운영 데이터 저장소. 현재는 Backend SG에서만 접근하며, AI의 읽기 전용 직접 접근은 설계가 결정되어 후속 반영 예정 (19장 참고)
VPC Endpoint (S3) + PrivateLink	목표 구조 · 미구현 — AI 서비스가 인터넷을 경유하지 않고 정책 문서 S3 버킷에 접근하는 경로

Amazon S3 (문서 버킷)	목표 구조 · 미구현 — 정책/FAQ/상품 문서 저장소. 현재는 정책 문서가 Markdown으로 AI 이미지에 baking됨
OpenAI API	AI 서비스가 NAT Gateway → Internet Gateway 경로로 호출하는 외부 LLM API

5. 네트워크 아키텍처 상세

VPC 구성

항목	값
VPC ID	vpc-078875ae5cd21cb7e
이름	hsa-vpc
CIDR 블록	10.0.0.0/16
DNS 지원 / DNS 호스트명	활성화 / 활성화

서브넷 구성

퍼블릭 서브넷 (Public Subnets)

서브넷 이름	서브넷 ID	AZ	CIDR	용도
hsa-public-subnet-1	subnet-04c74bf237c7313ec	us-east-1a	10.0.1.0/24	ALB, NAT Gateway
hsa-public-subnet-2	subnet-0c66662c3ef8184e6	us-east-1b	10.0.3.0/24	ALB 고가용성

라우팅: 0.0.0.0/0 → igw-0a69d751da3e355e4 (Internet Gateway 직접 연결)

프라이빗 서브넷 (Private Subnets)

서브넷 이름	서브넷 ID	AZ	CIDR	용도
hsa-private-subnet	subnet-0f99f5181cc513b7d	us-east-1a	10.0.2.0/24	ECS 서비스, RDS

라우팅 테이블 rtb-0e49e0596738005eb : 0.0.0.0/0 → nat-09517a4a532796db8 (NAT Gateway 경유)

참고

프라이빗 서브넷은 현재 us-east-1a 한 곳에만 구성되어 있어, ECS/RDS는 다중 AZ 이중화 상태는 아닙니다. PoC 단계에서는 비용 효율을 우선한 구성으로 보이며, 운영 단계 전환 시 멀티 AZ 프라이빗 서브넷 확장을 검토할 필요가 있습니다.

게이트웨이 구성

항목	값
Internet Gateway ID	igw-0a69d751da3e355e4
Internet Gateway 이름	hsa-igw
NAT Gateway ID	nat-09517a4a532796db8
NAT Gateway 이름	hsa-natgateway
NAT Gateway 위치	퍼블릭 서브넷 (subnet-04c74bf237c7313ec)
NAT Gateway 퍼블릭 IP	34.233.2.137
NAT Gateway 프라이빗 IP	10.0.1.207

6. 트래픽 플로우 상세

6-1. 프론트엔드 → 백엔드 API 플로우

```
사용자 브라우저
  ↓ (HTTPS)
Route 53 DNS (도메인 해석)
  ↓
Vercel CDN (프론트엔드 서빙)
  ↓ (API 호출)
Route 53 DNS (API 도메인 해석)
  ↓
Application Load Balancer (hsa-alb)
  ↓ (포트 8080)
ECS Backend Service (10.0.2.56:8080)
```

6-2. 백엔드 → AI 서비스 내부 통신 플로우

```
ECS Backend Service (10.0.2.56:8080)
  ↓ (DNS 조회: hsa-ai.hsa.local)
AWS Cloud Map (Service Discovery)
  ↓ (DNS 응답: 10.0.2.206)
ECS AI Service (10.0.2.206:8000)
  - 보안그룹: sg-013318a9a42ab13ea
  - 포트 8000만 백엔드 SG에서 허용
```

타임아웃 설정

RAG 경로(검색+재정렬+답변 생성)의 실측 최대 지연이 28.66초로 확인되어, Backend→AI 호출 타임아웃은 분류 LLM 호출과 네트워크 변동분을 더해 60초로 설정하기로 했습니다. 운영 중 모니터링 경고 기준은 타임아웃보다 먼저 지연 증가를 감지할 수 있도록 30초로 잡았습니다.

6-3. AI 서비스 → OpenAI API 플로우

```
ECS AI Service (10.0.2.206:8000)
  ↓ (아웃바운드 HTTPS)
프라이빗 서브넷 라우팅 테이블 (rtb-0e49e0596738005eb)
  ↓ (0.0.0.0/0 → NAT Gateway)
NAT Gateway (nat-09517a4a532796db8)
  - 프라이빗 IP: 10.0.1.207
  - 퍼블릭 IP: 34.233.2.137
  ↓
Internet Gateway (igw-0a69d751da3e355e4)
  ↓ (HTTPS/443)
OpenAI API (api.openai.com)
```

6-4. 백엔드 → RDS 데이터베이스 플로우

ECS Backend Service (10.0.2.56:8080)
↓ (PostgreSQL/5432)
RDS PostgreSQL (hsa-rds.cob42ieu6ara.us-east-1.rds.amazonaws.com:5432)
- 프라이빗 서브넷 내 위치
- 보안그룹: sg-06250f6a772c2a546 (Backend SG)

기술적 의사결정 메모 · RDS 접근 권한 변화

설계 문서 개정본의 이 항목에는 RDS 보안그룹으로 AI SG(sg-013318a9a42ab13ea)와 Backend SG가 함께 기재되어 있었는데, 같은 문서의 보안 그룹 설계 절에서는 "AI는 DB를 직접 조회하지 않으므로 RDS 접근에서 제외했다"고 설명하고 있어 처음에는 단순 오기로 보였습니다. 그런데 AI 파트 문서를 추가로 확인해보니, 실제로 AI가 RDS에 읽기 전용으로 직접 접근하는 방향으로 팀 결정이 있었던 것을 확인했습니다. 다만 이 결정은 아직 코드 단계에서 stub으로만 반영되어 있고 (rds_reader.py 가 항상 None 을 반환), DB 스키마와 읽기 전용 계정이 확정되지 않아 실제 보안그룹·IAM 변경은 이뤄지지 않은 상태였습니다. 즉 설계 문서의 두 구절은 둘 다 "맞는" 내용이었고, 하나는 현재 상태, 하나는 결정된 미래 상태를 가리키고 있었던 것입니다. 인프라 변경은 RDS 읽기 전용 계정과 스키마가 확정된 뒤, 보안그룹 인바운드 규칙에 AI SG를 추가하는 시점에 맞춰 진행할 계획입니다 (19장 참고).

6-5. 서비스 디스커버리 흐름

Backend Service → DNS Query (hsa-ai.hsa.local) → Cloud Map → IP Address (10.0.2.206) → AI Service

6-6. 외부 통신

방향	경로
Inbound	Internet Gateway → ALB → ECS Services
Outbound	ECS Services → NAT Gateway → Internet Gateway

7. ECS 컨테이너 서비스 상세

클러스터: hsa-cluster · 런치 타입 Fargate · 활성 서비스 2개 (hsa-backend, hsa-ai)

7-1. Backend 서비스 (hsa-backend)

속성	값
태스크 정의	hsa-back:39
런치 타입	Fargate
아키텍처	x86_64 / Linux
리소스	CPU 1024 (1 vCPU), 메모리 3072 MB
서브넷	subnet-0f99f5181cc513b7d (프라이빗, 퍼블릭 IP 비활성화)
보안그룹	sg-06250f6a772c2a546
현재 태스크 IP	10.0.2.56
포트	8080
로드밸런서 연결	ALB 타겟그룹 (hsa-backend-tg)
런타임	Spring Boot
이미지 태깅 규칙	ECR push 시 {git sha} 태그 부여 (배포마다 갱신)
애플리케이션 헬스체크	https://api.yourdomain.com/actuator/health
로그	CloudWatch /ecs/hsa-back

7-2. AI 서비스 (hsa-ai)

속성	값
태스크 정의	hsa-ai:8

런치 타입	Fargate
아키텍처	ARM64 (Graviton) / Linux
리소스	CPU 1024 (1 vCPU), 메모리 3072 MB
서브넷	subnet-0f99f5181cc513b7d (프라이빗, 퍼블릭 IP 비활성화)
보안그룹	sg-013318a9a42ab13ea
현재 태스크 IP	10.0.2.206
포트	8000
서비스 디스커버리	AWS Cloud Map 등록 (hsa-ai.hsa.local)
시크릿	AWS Secrets Manager (OPENAI_API_KEY)
환경변수 (적용 예정)	RAG_TOP_K=6, RAG_RERANK_TOP_N=3, RAG_RERANK_MODEL=gpt-5-nano
헬스체크	ECS: /health 엔드포인트, 30초 간격 · Cloud Map: 커스텀 헬스체크 (실패 임계값 1)
이미지 태깅 규칙	PR head_sha 앞 12자 기준 ECR push (merge 전 커밋과 1:1 추적)
로그	CloudWatch /ecs/hsa-ai

환경변수 행은 RAG 검색/재정렬 품질 튜닝을 위해 코드에서는 이미 사용 중이지만, ECS Task Definition에는 아직 반영 전인 항목입니다. 다음 배포 때 함께 적용할 계획입니다.

참고 · Fargate 태스크 IP의 가변성

ECS Fargate 태스크의 프라이빗 IP는 재배포(롤링 디플로이) 시마다 새 ENI가 할당되며 바뀔 수 있습니다. 초기 설계 스냅샷에는 AI 서비스 IP가 10.0.2.18로 기록되어 있었으나, 이후 갱신된 기준으로 현재 IP는 10.0.2.206입니다. 이는 오류가 아니라 그 사이 재배포가 발생했기 때문이며, 본 문서는 가장 최신 값을 기준으로 작성했습니다. 마찬가지로 태스크 정의 리비전도 hsa-back:3 → hsa-back:39, hsa-ai:6 → hsa-ai:8로 갱신되어 다수의 배포가 누적되었음을 보여줍니다.

8. 서비스 디스커버리 (AWS Cloud Map)

네임스페이스 구성

속성	값
네임스페이스	hsa.local
타입	DNS_PRIVATE (프라이빗 DNS)
네임스페이스 ID	ns-n6j36y6rlqmuzbz7
호스팅 존 ID	Z016581831EJM312NC3LT
DNS SOA TTL	15초
생성일	2026-05-16 11:21:15 UTC

등록된 서비스: hsa-ai

속성	값
서비스 ID	srv-tks3uwsptsttgrg
타입	DNS_HTTP
DNS 구성	A 레코드 (IPv4), TTL 60초, MULTIVALUE 라우팅
헬스체크	커스텀 헬스체크 (실패 임계값: 1)
DNS 이름	hsa-ai.hsa.local
현재 인스턴스	10.0.2.206 (HEALTHY)

```
# Backend에서 AI 서비스 호출 예시
import requests
ai_service_url = "http://hsa-ai.hsa.local:8000"
response = requests.post(f"{ai_service_url}/api/chat", json=payload)
```

9. 보안 그룹 설계

보안 점검 및 개선

초기 구성을 점검하던 중 ALB SG에 5432 (PostgreSQL), 8080 (Backend 직접 노출) 인바운드가 0.0.0.0/0으로 열려 있고, Backend SG에도 5432 인바운드가 0.0.0.0/0으로 열려 있는 것을 발견해 제거했습니다. 둘 다 불필요한 외부 노출이었습니다. ALB는 80/443으로만 받아 타겟그룹을 통해 8080으로 포워딩하면 충분하고, Backend는 RDS로 나가는 아웃바운드만 필요하며 인바운드로 5432를 받을 이유가 없습니다. RDS는 별도 보안그룹을 두어 Backend SG에서만 5432 인바운드를 허용하도록 정리했고, 현재 AI는 DB를 직접 조회하지 않으므로 RDS 접근 권한에서 제외했습니다 (다만 AI의 RDS 읽기 전용 직접 접근이 팀 결정 사항으로 확정되어 있어, 후속 변경이 예정되어 있습니다 — 19장 참고).

9-1. ALB 보안 그룹 (sg-06df1af7c2676558b)

인바운드 규칙

프로토콜	포트	소스	설명
HTTP	80	0.0.0.0/0	웹 트래픽
HTTPS	443	0.0.0.0/0	보안 웹 트래픽

아웃바운드 규칙

프로토콜	포트	대상	설명
TCP	8080	sg-06250f6a772c2a546	타겟그룹(Backend)으로 포워딩

[제거됨] TCP 5432 ← 0.0.0.0/0, TCP 8080 ← 0.0.0.0/0 인바운드 (불필요한 외부 노출)

9-2. Backend 보안 그룹 (sg-06250f6a772c2a546)

인바운드 규칙

프로토콜	포트	소스	설명
TCP	8080	sg-06df1af7c2676558b	ALB에서만 접근

아웃바운드 규칙

프로토콜	포트	대상	설명
All	All	0.0.0.0/0	RDS, AI 서비스 등 모든 아웃바운드 트래픽

[제거됨] TCP 5432 ← 0.0.0.0/0 인바운드 (불필요한 외부 노출)

9-3. AI 서비스 보안 그룹 (sg-013318a9a42ab13ea)

인바운드 규칙

프로토콜	포트	소스	설명
TCP	8000	sg-06250f6a772c2a546	Backend에서만 접근

아웃바운드 규칙

프로토콜	포트	대상	설명
All	All	0.0.0.0/0	OpenAI API 호출용 (NAT Gateway 경유)

9-4. RDS 보안 그룹

인바운드 규칙

프로토콜	포트	소스	설명
TCP	5432	sg-06250f6a772c2a546	Backend에서만 접근. AI SG는 제외

아웃바운드 규칙

프로토콜	포트	대상	설명
All	All	0.0.0.0/0	기본값 (필요 시 제한 가능)

남은 확인 사항

RDS 보안그룹의 실제 ID는 설계 문서에 명시적으로 기록해두지 않아 콘솔에서 다시 확인해야 합니다. AI SG를 RDS에서 제외한 이유와 향후 변경 계획은 6-4절과 19장에 정리했습니다.

9-5. 보안 그룹 요약 흐름

ALB(80,443) → Backend SG(8080) → AI SG(8000)
↓
RDS SG(5432) ← Backend SG만 허용 (AI 제외)

10. 컨테이너 레지스트리 (ECR)

이름	URI	생성일	암호화
hsa-ai	325899476013.dkr.ecr.us-east-1.amazonaws.com/hsa-ai	2026-05-16	AES256
hsa-backend	325899476013.dkr.ecr.us-east-1.amazonaws.com/hsa-backend	2026-05-16	AES256

11. 로드 밸런서 (ALB)

항목	값
이름	hsa-alb
DNS	hsa-alb-1734268684.us-east-1.elb.amazonaws.com
스킴	Internet-facing
서브넷	Public 서브넷 (us-east-1a, us-east-1b)
보안 그룹	hsa-alb-sg (sg-06df1af7c2676558b)
ALB ARN	arn:aws:elasticloadbalancing:us-east-1:325899476013:loadbalancer/app/hsa-alb/ebbfd3d397fc78ed
타겟그룹 ARN	arn:aws:elasticloadbalancing:us-east-1:325899476013:targetgroup/hsa-backend-tg/6ed7f089a0dadabd

12. 데이터베이스 (RDS) 상세

항목	값
인스턴스 ID	hsa-rds
엔진	PostgreSQL 15.18
인스턴스 클래스	db.t3.micro
엔드포인트	hsa-rds.cob42ieu6ara.us-east-1.rds.amazonaws.com:5432
가용 영역	us-east-1a
스토리지	200GB (gp3, 자동 확장 최대 1TB)
백업 보존	7일
암호화	KMS 암호화 활성화
Performance Insights	7일 보존
마스터 사용자명	hsa_**** (마스킹 처리됨)
접근 가능한 보안그룹	Backend SG (sg-06250f6a772c2a546)만 허용

13. IAM 및 보안 (KMS 포함)

13-1. ECS 실행 역할

항목	값
역할명	hsa-ecs-execution-role
관리형 정책	AmazonECSTaskExecutionRolePolicy AmazonSSMManagedInstanceCore AmazonECS_FullAccess AmazonElasticContainerRegistryPublicFullAccess
인라인 정책	ssmessage (SSM Session Manager)

13-2. KMS 암호화

- S3 기본 암호화 키
- RDS 기본 암호화 키

13-3. CI/CD 배포 권한

Backend 배포 워크플로는 `configure-aws-credentials` 액션에 시크릿 `AWS_ROLE_TO_ASSUME` 으로 저장된 역할 ARN을 전달해 임시 자격증명을 발급받습니다. 이 역할의 실제 이름은 `hsa-deploy-role` 입니다. AI 서비스 배포 워크플로는 같은 액션에 `AWS_OIDC_ROLE_ARN` 이라는 별도 시크릿을 전달합니다. 시크릿 이름이 다른 것으로 보아 두 서비스가 서로 다른 IAM 역할을 사용하는 것으로 보이며, 둘 다 GitHub OIDC 공급자를 신뢰하는 역할을 Assume하는 방식이라 장기 액세스 키를 코드/시크릿에 저장하지 않습니다. AI 배포 워크플로에는 다음과 같이 필요한 권한만 명시적으로 선언되어 있습니다.

```
permissions:
  id-token: write      # OIDC로 AWS 임시 자격증명을 받기 위해 필요
  contents: write      # squash merge를 위해 필요
  pull-requests: write # gh pr merge를 위해 필요
```

남은 확인 사항

두 시크릿이 실제로 가리키는 IAM 역할의 이름과 신뢰 정책(trust policy)은 GitHub Secrets/IAM 콘솔에만 있어 이 문서에는 값 자체를 적지 않았습니다. 보안상 시크릿 값을 문서에 노출하지 않는 것이 맞다고 판단했고, 대신 권한 분리가 되어 있다는 사실과 그 근거(시크릿 이름 차이, 워크플로 permissions 블록)만 기록해 두었습니다.

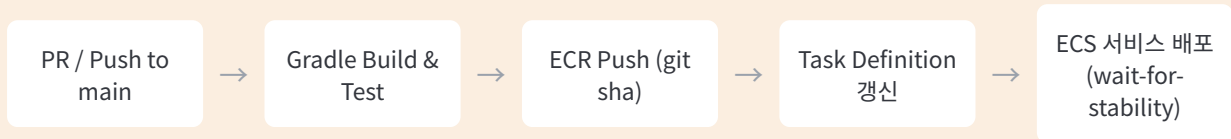
14. 배포 파이프라인 상세 (CI/CD)

세 서비스(Frontend / Backend / AI)는 각각 독립적인 배포 파이프라인을 가지며, Backend와 AI는 수동 CLI 배포가 아니라 **GitHub Actions 기반 CI/CD**로 운영됩니다.

Frontend (Vercel)

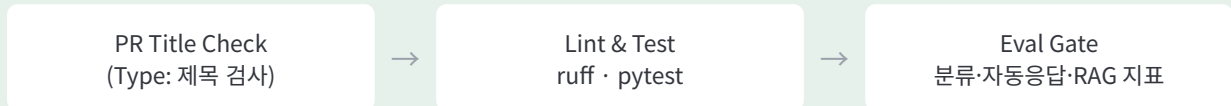


Backend · hsa-back (main 브랜치)



AI 서비스 · hsa-ai (development → main)

① feature → development



② development → main

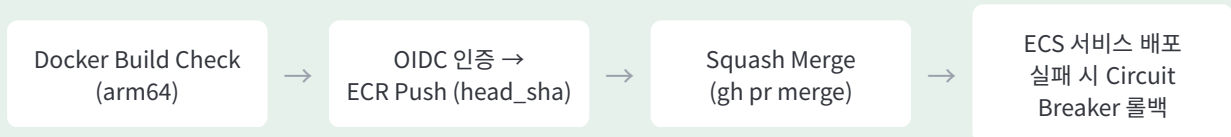


그림 2. HSA CI/CD 파이프라인 (서비스별 배포 흐름)

14-1. 프론트엔드 배포 (Vercel)

```
# Vercel CLI 설치
npm i -g vercel

# 프로젝트 초기화
vercel

# 환경 변수 설정
vercel env add NEXT_PUBLIC_API_URL production
# 값: https://api.yourdomain.com (Route 53으로 설정된 API 도메인)

# 배포
vercel --prod
```

자동 배포 설정 — Git 연동: GitHub 저장소 연결 · 브랜치 전략: main → Production 배포, develop → Preview 배포 · 빌드 명령어: npm run build

14-2. Backend 서비스 배포 (GitHub Actions)

main 브랜치 기준 PR/Push 시 자동으로 빌드·배포되는 단일 워크플로입니다. 트리거는 `pull_request` 또는 `push to main` 입니다.

[CI] build job

1. JDK 21(Temurin) 세팅, Gradle 캐시 적용
2. `./gradlew clean build -x test` 로 빌드 (hsa-core 디렉토리)
3. 빌드된 jar를 artifact로 업로드 (1일 보존)

[CD] deploy job (push 이벤트일 때만, build 성공 후 실행)

1. AWS 자격증명 구성 (configure-aws-credentials, role-to-assume)
2. Amazon ECR 로그인
3. Docker 이미지 빌드 후 ECR에 {git sha} 태그로 푸시
4. 현재 등록된 ECS 태스크 정의 다운로드
5. 새 이미지 URI로 태스크 정의 렌더링
6. amazon-ecs-deploy-task-definition 액션으로 ECS 서비스(hsa-backend) 롤링 배포, wait-for-service-stability: true로 배포 완료까지 대기

```
# .github/workflows/deploy.yml (요약)
name: HSA Server CI/CD Pipeline
on:
  pull_request:
    branches: ["main"]
  push:
    branches: ["main"]
jobs:
  build:
    # JDK 21 + Gradle build, jar artifact 업로드
  deploy:
    needs: build
    if: github.event_name == 'push'
    # ECR push → task definition 갱신 → ECS 배포
```

14-3. AI 서비스 배포 (GitHub Actions)

AI 서비스는 development / main 두 브랜치를 대상으로 CI 단계가 분기되는 통합 워크플로(`ci.yml`)와, 실제 배포를 담당하는 재사용 워크플로(`deploy.yml`)로 구성되어 있습니다.

전환	실행되는 Job
feature → development	PR 제목 검사 · Lint/Test · Eval Gate (병렬)
development → main	PR 제목 검사 · Lint/Test → Docker Build Check(arm64) → Deploy

1) PR Title Check — 모든 PR에서 커밋/PR 제목이 `Type: 제목` 컨벤션을 따르는지 검사

2) Lint & Test

```
ruff check .
HSA_TEST_MODE=true pytest -q
```

3) Eval Gate (development 대상 PR만)

- API 서버를 uvicorn으로 직접 기동 후 `/health` 확인
- `python evals/main.py` 로 분류 정확도, 자동응답 정확도, Pydantic 검증, RAG 근거 일치율 등 4개 지표 게이트 실행
- 실제 OpenAI API를 호출하므로 비용이 발생 (OPENAI_API_KEY 시크릿 필요)

4) Docker Build Check (main 대상 PR만)

- QEMU + Buildx로 linux/arm64 빌드만 검증 (push 없음)
- 배포 직전이 아니라 PR 단계에서 ARM64 빌드 실패를 미리 잡기 위한 게이트

5) Deploy (deploy.yml, workflow_call로 호출)

- main 대상 PR이 위 게이트를 모두 통과해야 실행
- 배포 순서가 일반적인 "merge 후 배포"가 아니라 "ECR push 먼저 → squash merge → ECS 배포" 순서로 설계되어, merge 이후 실패 지점이 ECS 배포 단계 하나로 줄어들
- 이미지 태그는 PR head_sha 앞 12자(merge 전 커밋) 기준 → ECR 태그와 PR이 1:1로 추적됨

`concurrency: group: deploy-main, cancel-in-progress: false` 로 배포를 직렬화 — 동시에 머지된 PR이 있어도 배포가 겹치지 않고, 진행 중인 배포를 취소하지 않은 채 순서대로 처리

1. PR 브랜치 체크아웃 (`head_sha` 기준, merge 전)
2. GitHub OIDC로 AWS 임시 자격증명 발급 (장기 액세스 키 미사용)
3. Amazon ECR 로그인 및 이미지 push
4. PR squash merge (`gh pr merge`)
5. 현재 ECS 태스크 정의 다운로드 → 이미지만 교체
6. ECS 서비스 롤링 배포 (`wait-for-service-stability: true`)
 - 새 태스크가 헬스체크 통과 전까지 기존 태스크 유지 → 무중단 배포
 - 헬스체크 실패 시 ECS Circuit Breaker가 이전 revision으로 자동 롤백

배포 실패 시 상태

main 코드는 새 버전이지만 ECS는 이전 버전으로 롤백된 상태가 됩니다. 복구는 development에서 빈 커밋을 push해 PR을 재생성하고 파이프라인을 재실행하는 방식입니다.

```
# .github/workflows/ci.yml + deploy.yml (요약)
jobs:
  title-check:      # 모든 PR
  lint-and-test:    # 모든 PR
  eval:             # development 대상 PR만
  docker-build-check: # main 대상 PR만, arm64
  call-deploy:      # main 대상 PR만, deploy.yml 호출
    uses: ../github/workflows/deploy.yml
    with:
      pr_number: ${ github.event.pull_request.number }
      head_sha:  ${ github.event.pull_request.head.sha }
```

15. OpenAI API 연동 구성

15-1. NAT Gateway를 통한 외부 API 호출

```
import openai
import boto3
from botocore.exceptions import ClientError

def get_openai_api_key():
    """AWS Secrets Manager에서 OpenAI API 키 조회"""
    secret_name = "openai-api-key"
    region_name = "us-east-1"

    session = boto3.session.Session()
    client = session.client(
        service_name='secretsmanager',
        region_name=region_name
    )

    try:
        get_secret_value_response = client.get_secret_value(
            SecretId=secret_name
        )
        return get_secret_value_response['SecretString']
    except ClientError as e:
        raise e

openai.api_key = get_openai_api_key()

# NAT Gateway (34.233.2.137)를 통해 api.openai.com 호출
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": "Hello"}]
)
```

15-2. 네트워크 경로 추적

```
AI Container (10.0.2.206)
→ Private Route Table (rtb-0e49e0596738005eb)
→ NAT Gateway (10.0.1.207 → 34.233.2.137)
→ Internet Gateway (igw-0a69d751da3e355e4)
→ Internet → api.openai.com
```

16. DNS 및 도메인 관리 (Route 53)

실제 도메인명은 비공개 처리되어 예시로 yourdomain.com을 사용합니다.

16-1. 호스팅 존 구성

호스팅 존: yourdomain.com

네임서버:

- ns-xxx.awsdns-xx.com
- ns-xxx.awsdns-xx.co.uk
- ns-xxx.awsdns-xx.net
- ns-xxx.awsdns-xx.org

16-2. DNS 레코드 설정

A. 프런트엔드 도메인 (Vercel)

레코드 타입	이름	값	TTL
CNAME	www	cname.vercel-dns.com	300
CNAME	@ (apex)	alias.vercel-dns.com	300

남은 확인 사항 · Zone Apex CNAME 표기

DNS 표준(RFC 1034/1035)상 zone apex(@)에는 일반 CNAME 레코드를 사용할 수 없습니다. Route 53에서 apex 도메인에 Vercel 같은 외부 호스트를 연결할 때는 보통 Route 53의 Alias 레코드(타입 A)를 사용합니다. 설계 문서에는 apex 레코드가 "CNAME" 타입으로 적혀 있어, 실제로는 Alias 레코드인지 혹은 Vercel 측에서 별도의 ANAME/플래트닝 방식을 쓰는지 콘솔에서 다시 확인할 계획입니다.

B. API 도메인 (ALB)

레코드 타입	이름	값	TTL
A (Alias)	api	hsa-alb-1734268684.us-east-1.elb.amazonaws.com	-
A (Alias)	api-dev	hsa-alb-1734268684.us-east-1.elb.amazonaws.com	-

```
# 호스팅 존 생성
aws route53 create-hosted-zone \
  --name yourdomain.com \
  --caller-reference $(date +%s)

# 프론트엔드 CNAME 레코드 생성
aws route53 change-resource-record-sets \
  --hosted-zone-id Z1234567890ABC \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name": "www.yourdomain.com",
        "Type": "CNAME",
        "TTL": 300,
        "ResourceRecords": [{"Value": "cname.vercel-dns.com"}]
      }
    }]
  }'
```

```
# API Alias 레코드 생성
aws route53 change-resource-record-sets \
  --hosted-zone-id Z1234567890ABC \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name": "api.yourdomain.com",
        "Type": "A",
        "AliasTarget": {
          "DNSName": "hsa-alb-1734268684.us-east-1.elb.amazonaws.com",
          "EvaluateTargetHealth": false,
          "HostedZoneId": "Z35SXD0TRQ7X7K"
        }
      }
    }]
  }'
```

16-3. SSL/TLS 인증서 설정

Vercel (프론트엔드) — 자동 SSL: Vercel이 Let's Encrypt를 통해 자동 발급 · 커스텀 도메인: Vercel 대시보드에서 도메인 추가 · HTTPS 리디렉션: 자동 활성화

ALB (백엔드 API)

```
# ACM에서 SSL 인증서 요청
aws acm request-certificate \
  --domain-name api.yourdomain.com \
  --subject-alternative-names api-dev.yourdomain.com \
  --validation-method DNS \
  --region us-east-1

# ALB 리스너에 HTTPS 추가
aws elbv2 create-listener \
  --load-balancer-arn arn:aws:elasticloadbalancing:us-east-1:325899476013:loadbalancer/app/hsa-alb/ebbfd3d397fc78ed \
  --protocol HTTPS \
  --port 443 \
  --certificates CertificateArn=arn:aws:acm:us-east-1:325899476013:certificate/xxx \
  --default-actions Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:us-east-1:325899476013:targetgroup/hsa-backend-tg/6ed7f089a0dadabd
```

16-4. 도메인 구조 예시

```
yourdomain.com (프론트엔드 - Vercel)
├─ www.yourdomain.com (프론트엔드 - Vercel)
├─ api.yourdomain.com (백엔드 API - ALB)
└─ api-dev.yourdomain.com (개발 환경 API - ALB)
```

17. 모니터링 및 로깅

17-1. CloudWatch 로그 그룹

서비스	로그 그룹
Backend	/ecs/hsa-back
AI Service	/ecs/hsa-ai

17-2. 헬스체크 엔드포인트

서비스	엔드포인트
Backend	https://api.yourdomain.com/actuator/health
AI Service	http://10.0.2.206:8000/health (내부 통신용)
Frontend	https://yourdomain.com (Vercel 자동 모니터링)

17-3. Vercel 모니터링

```
# Vercel 함수 로그 확인
vercel logs

# 특정 배포 로그 확인
vercel logs --url=https://yourdomain.com
```

18. 운영 고려사항

18-1. 비용 최적화

- Vercel: Hobby 플랜(무료) 또는 Pro 플랜(\$20/월)
- AI 서비스: ARM64 Graviton 프로세서로 비용 절감
- Route 53: 호스팅 존(\$0.50/월) + 쿼리 비용
- NAT Gateway: 시간당 요금 + 데이터 처리 요금

18-2. 보안

- OpenAI API 키는 AWS Secrets Manager로 관리
- HTTPS는 모든 도메인에 적용
- 보안 그룹은 ALB(80/443) → Backend(8080) → AI(8000) → RDS(5432) 순으로 최소 권한만 허용하도록 설계 (9장 참고)
- CORS는 프론트엔드 도메인만 허용하도록 백엔드 설정

18-3. 가용성 및 성능

- Vercel CDN: 글로벌 Edge 네트워크
- ALB: 다중 AZ 구성 (us-east-1a, us-east-1b)
- NAT Gateway: 현재 단일 구성(Single NAT) — 해당 AZ 장애 시 AI 서비스의 OpenAI API 호출(아웃바운드)이 끊길 수 있어, 운영 단계에서는 AZ별 NAT Gateway 이중화를 검토할 필요가 있음
- ECS 배포: Circuit Breaker 기반 자동 롤백으로 무중단 배포 보장
- Backend→AI 호출 타임아웃: 실측 지연 데이터를 기반으로 60초로 설정 (6-2절 참고)

18-4. 확장성

- Vercel: 자동 스케일링(서버리스)
- ECS Fargate: CPU/메모리 기반 자동 스케일링
- RDS: 스토리지 자동 확장(최대 1TB)

19. 트러블슈팅 및 기술적 의사결정

인프라를 구축하고 운영하면서 직접 발견하고 해결한 문제, 그리고 아직 진행 중인 의사결정을 정리합니다. 단순히 구성을 나열하는 것보다 "왜 이렇게 바꿨는지"를 남기는 것이 나중에 비슷한 문제를 만났을 때 더 유용하다고 생각해 기록했습니다.

19-1. 직접 발견하고 수정한 보안 이슈

- ① **ALB/Backend 보안그룹의 불필요한 외부 노출 제거** — 인프라 점검 중 ALB SG에 5432/8080 포트가, Backend SG에 5432 포트가 각각 0.0.0.0/0으로 열려 있는 것을 발견했습니다. ALB는 80/443만 받아 타겟그룹으로 포워딩하면 되고 DB 포트를 열 이유가 없었기 때문에 제거했습니다 (9장 참고).
- ② **RDS 전용 보안그룹 분리** — 기존에는 RDS 접근 제어가 Backend SG 설정에 뒤섞여 있어 가독성이 떨어졌습니다. RDS 전용 보안그룹을 새로 만들어 Backend SG에서만 5432 인바운드를 허용하도록 분리했습니다.
- ③ **프론트엔드 구조 확정 반영** — 초기에는 S3 정적 호스팅도 검토되었으나, 최종적으로 Vercel + Route 53 구조로 확정되어 인프라 문서와 다이어그램에 반영했습니다.

19-2. 인프라 변경 이력 추적

ECS Fargate는 재배포(롤링 디플로이)마다 태스크에 새 ENI가 할당되어 프라이빗 IP가 바뀝니다. 처음에는 이걸 모르고 문서에 적힌 IP가 실제와 다르길래 "오류인가?" 하고 헛갈렸는데, 태스크 정의 리비전(hsa-back:3→39, hsa-ai:6→8)이 계속 올라가 있는 걸 보고서야 단순히 그 사이 여러 번 재배포가 있었다는 걸 알았습니다. 이후로는 인프라 문서에 IP나 리비전 같은 값을 적을 때 "이 값은 재배포 시 바뀔 수 있다"는 점을 함께 표시하는 습관이 생겼습니다 (7장 참고).

19-3. 진행 중인 기술적 의사결정

[A] AI의 RDS 직접 접근 — 결정은 됐지만 아직 미반영

설계 문서를 다시 보다가 "RDS는 Backend SG에서만 접근 가능"이라는 문장과 "RDS 보안그룹에 AI SG도 포함된다"는 문장이 서로 다른 절에 동시에 적혀 있는 걸 발견했습니다. 처음엔 제가 어딘가에서 잘못 옮겨적었나 싶었는데, AI 파트 쪽 설계 문서를 다시 확인해보니 실제로 AI가 RDS에 읽기 전용으로 직접 접근하도록 팀 결정이 이미 내려진 상태였고, 다만 DB 스키마와 읽기 전용 계정이 아직 확정되지 않아 코드에서는 stub으로만 처리되어 있었습니다. 즉 인프라(보안그룹)는 아직 예전 상태고, 애플리케이션 설계는 이미 다음 단계로 넘어가 있었던 것입니다. 읽기 전용 계정/스키마가 확정되면 RDS SG 인바운드에 AI SG를 추가하는 작업을 진행할 계획입니다.

[B] Zone Apex에 CNAME 레코드 사용 표기

DNS 표준상 zone apex에는 일반 CNAME을 사용할 수 없어, Route 53의 Alias(A) 레코드 사용이 일반적입니다. 설계 문서에는 apex 레코드가 CNAME 타입으로 기재되어 있어, 실제 Route 53 콘솔 설정을 다시 확인해 필요하면 Alias 레코드로 정정할 계획입니다.

[C] IAM 배포 역할 분리 확인

Backend와 AI 배포 워크플로가 서로 다른 이름의 GitHub Secret(AWS_ROLE_TO_ASSUME / AWS_OIDC_ROLE_ARN)을 사용하는 것을 보고 두 서비스가 별도의 IAM 역할로 배포 권한이 분리되어 있다는 것을 확인했습니다. 다만 각 역할의 정확한 신뢰 정책 내용까지는 문서로 남기지 않아, 필요할 때 IAM 콘솔에서 다시 확인하기로 했습니다 (13장 참고).

[D] RDS 보안그룹 ID 미기록

작업 당시 RDS 보안그룹의 규칙 내용은 정리했지만 실제 보안그룹 ID를 따로 기록해두지 않았습니다. 리소스 ID는 사소한 정보라도 나중에 추적할 때 꼭 필요하다는 걸 다시 느꼈고, 이후로는 보안그룹을 만들 때마다 ID를 바로 메모해 두는 습관을 들였습니다.

20. 배운 점 및 향후 개선 방향

20-1. 배운 점

- 보안 그룹은 처음부터 최소 권한으로 설계하는 게 결국 더 빠르다. ALB에 DB 포트를 열어둔 것처럼 "일단 되게 만" 만들어두면, 나중에 그걸 찾아서 정리하는 비용이 처음 제대로 설계하는 것보다 훨씬 큼니다.
- Fargate 환경에서는 태스크 IP를 고정값으로 취급하면 안 된다. 재배포마다 IP가 바뀐다는 걸 직접 겪어보고서야, Cloud Map 같은 서비스 디스커버리가 왜 필요한지 체감했습니다.
- 설계 문서와 실제 구현 사이에는 항상 시간차가 있다. 문서의 두 문장이 서로 다른 말을 하고 있을 때, 무조건 한 쪽이 틀렸다고 단정하기보다 "혹시 하나는 현재고 하나는 계획인가?"를 먼저 확인하는 습관이 생겼습니다.
- 리소스 ID, 시크릿 이름처럼 작은 값도 그때그때 기록해야 한다. 나중에 다시 찾으려면 콘솔을 한참 뒤져야 하는데, 그 시간이 생각보다 아깝습니다.

20-2. 향후 개선 방향

- 프라이빗 서브넷을 us-east-1a 한 곳에서 다중 AZ로 확장해 ECS/RDS 이중화 적용
- NAT Gateway를 AZ별로 이중화해 단일 NAT 장애 시 AI→OpenAI 아웃바운드가 끊기지 않도록 개선
- AI의 RDS 읽기 전용 접근이 확정되면 보안그룹·IAM 정책에 반영하고 본 문서 업데이트
- S3 문서 버킷 + VPC Endpoint/PrivateLink 구조를 실제로 구현해 정책 문서를 이미지 baking 방식에서 분리
- RDS 보안그룹 등 누락된 리소스 ID를 콘솔에서 재확인해 문서에 채워넣기
- ECS Task Definition에 RAG 관련 환경변수(RAG_TOP_K 등) 반영

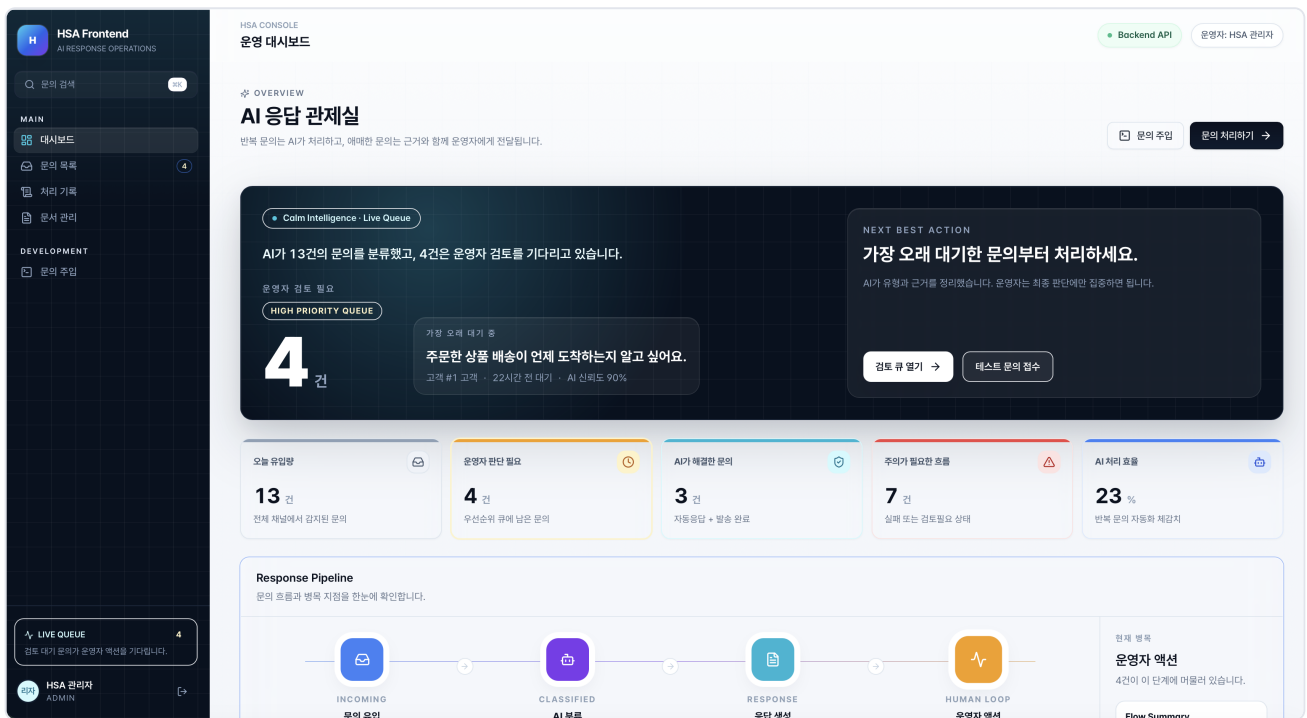
이 문서는 HSA 프로젝트 AWS 인프라 구축 과정을 기준으로 2026-06-23에 작성되었습니다. 리전: us-east-1 · 계정: 325899476013 · 민감 정보(비밀번호, 계정 일부)는 마스킹 처리했습니다.

21. 프로젝트 결과 화면 (Frontend 실행 화면)

아래는 PoC 단계에서 실제로 동작하는 HSA Frontend(운영자 콘솔) 화면입니다. Backend API와 연동되어 문의 수집 → AI 분류/판단 → 운영자 검토 → 처리 기록까지의 흐름을 한 화면에서 확인할 수 있도록 구성했습니다. AI 서버는 프론트엔드와 직접 통신하지 않고, 항상 Backend를 경유해 결과를 전달합니다 (1장 참고).

대시보드 — AI 응답 관제실

전체 문의 유입량, 운영자 판단이 필요한 건수, AI가 자체적으로 해결한 문의, 처리 효율을 한눈에 보여주는 운영 현황판입니다. 가장 오래 대기 중인 문의를 상단에 노출해 운영자가 우선순위를 빠르게 파악할 수 있도록 했습니다.



문의 목록 — AI 검토함

AI가 분류·신뢰도 판단을 마친 문의 중 운영자 검토가 필요한 항목만 모아 보여주는 큐(Queue) 화면입니다. 채널·유형별 필터링과 AI 판단·신뢰도·다음 액션을 함께 노출해, 운영자가 근거를 확인한 뒤 빠르게 발송 여부를 결정할 수 있도록 설계했습니다.

HSA Frontend
AI RESPONSE OPERATIONS

문의 검색

MAIN

대시보드

문의 목록

처리 기록

문서 관리

DEVELOPMENT

문의 주입

LIVE QUEUE

HSA 관리자
ADMIN

HSA CONSOLE

문의 처리함

Backend API

운영자: HSA 관리자

REVIEW INBOX

AI 검토함

검토가 필요한 문의부터 빠르게 확인하고 응답을 마무리하세요.

고객명, 문의 ID, 본문 검색

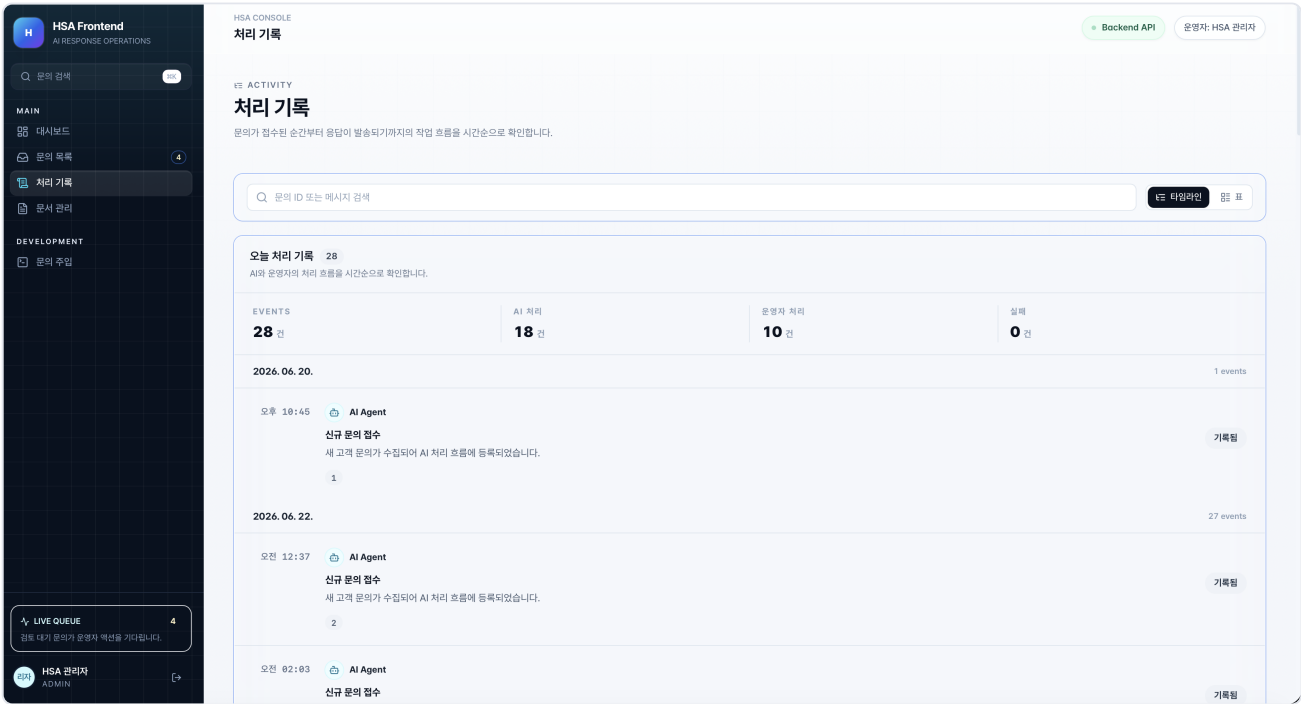
상태 전체 검토 필요 자동응답 발송 완료 채널 전체 카카오톡 인스타 이메일 유형 전체 배송 교환/반품 상품 기타

우선순위로 J / K 이동 열거 / 검색

문의	AI 판단	신뢰도	상태	다음 액션	대기
IO 주문한 상품 배송이 언제 도착하는지 알고 싶어요. 13 고객 #6 고객 주문 배송 상태와 예상 도착일을 문의했습니다.	배송 배송 · 주문 식별 필요	98% 높음 근거 확인됨	초안완료 수동처리	초안 확인 후 발송 →	18시간 전
IG QA 확인용 질 문의입니다. 배송 상태가 궁금합니다. 12 고객 #6 고객 주문 배송 상태와 예상 도착일을 문의했습니다.	배송 배송 · 주문 식별 필요	98% 높음 근거 확인됨	초안완료 수동처리	초안 확인 후 발송 →	18시간 전
E 환불하고싶어요 9 고객 #5 고객 수령한 상품의 뒷/사이즈 차이로 교환·환불 가능 여부를 문의했습니다.	교환/반품 교환/반품 · 정책 근거 필요	90% 높음 자율화 가능	인도필요 수동처리	환불 조건 확인 →	19시간 전
K 주문한 상품 배송이 언제 도착하는지 알고 싶어요. 8 고객 #1 고객 주문 배송 상태와 예상 도착일을 문의했습니다.	배송 배송 · 주문 식별 필요	90% 높음 근거 확인됨	초안완료 수동처리	초안 확인 후 발송 →	22시간 전

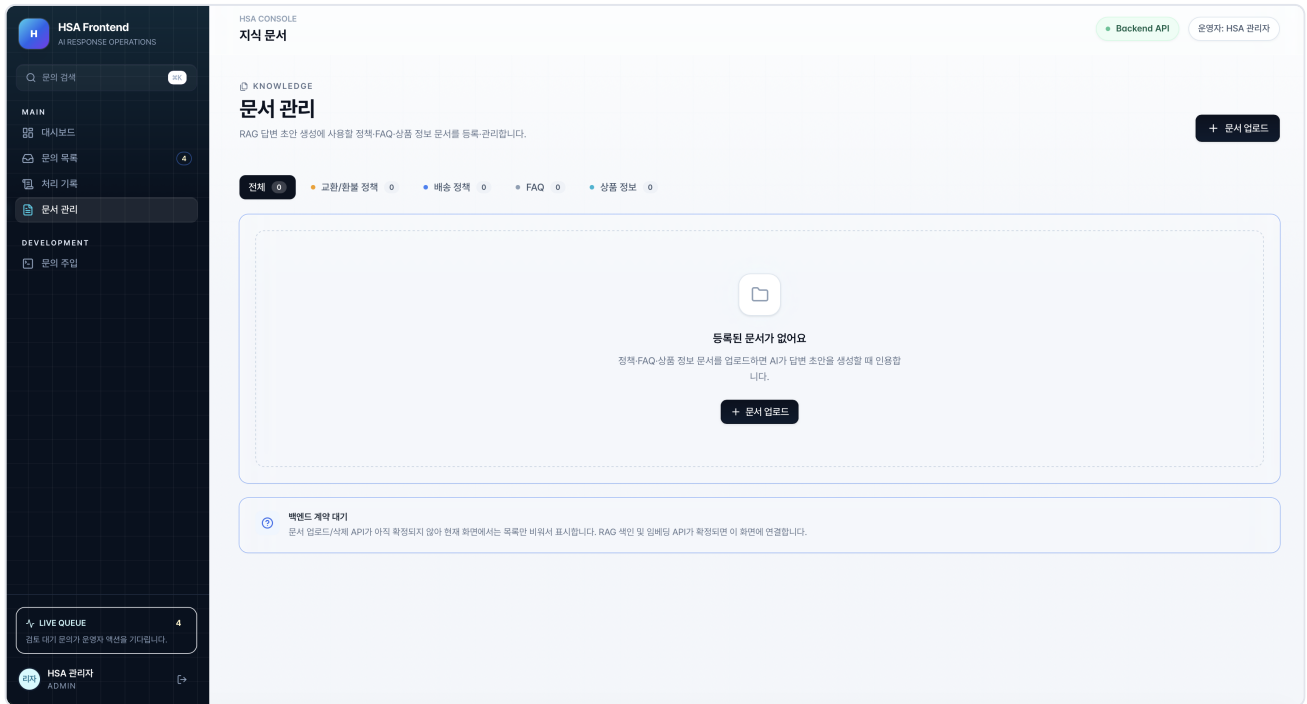
처리 기록 — Activity Timeline

문의가 접수된 순간부터 응답이 발송되기까지의 전체 처리 흐름을 시간순으로 기록한 화면입니다. AI 처리 건수와 운영자 처리 건수, 실패 건수를 일별로 집계해 보여주어, AI 자동화가 실제로 운영 부담을 얼마나 줄이고 있는지 추적할 수 있습니다.



문서 관리 — 지식 문서(RAG 소스)

RAG 답변 초안 생성에 사용할 배송 정책, 교환/환불 정책, FAQ, 상품 정보 문서를 업로드·관리하는 화면입니다. 문서 업로드/삭제 API와 RAG 색인·임베딩 파이프라인이 Backend와 확정되는 시점에 맞춰 연결할 예정이며, 현재는 PoC 단계라 화면 골격만 우선 구현해 둔 상태입니다.



문의 접수 테스트 — 개발용 도구

실제 채널(카카오톡·인스타그램·이메일) 연동 전, Backend 웹훅으로 문의가 정상적으로 접수·저장·AI 처리되는지 검증하기 위한 개발/테스트 전용 화면입니다. 운영 환경에서는 비활성화되며, 이 화면으로 생성한 문의도 일반 문의 목록과 동일한 파이프라인으로 처리됩니다.

HSA Frontend
AI RESPONSE OPERATIONS

문의 검색

MAIN

대시보드

문의 목록4

처리 기록

문서 관리

DEVELOPMENT

문의 주입

LIVE QUEUE4
검토 대기 문의가 운영자 역션을 기다립니다.

HSA 관리자
ADMIN

HSA CONSOLE

문의 접수 테스트

DEVELOPMENT TOOL

문의 접수 테스트

실제 채널 연동 전, 백엔드 웹훅으로 문의 접수를 검증합니다.

PoC 개발/테스트 전용 화면
실제 서비스에서는 노출되지 않으며, 운영 환경에서 비활성화됩니다. 이 화면으로 생성된 문의는 일반 문의 목록과 동일하게 처리됩니다.

테스트 문의 접수

채널과 본문을 입력하면 백엔드 웹훅으로 문의를 접수한 뒤 상세 화면으로 이동합니다.

채널

K 카카오
비즈니스 채팅

인스타
DM

이메일
CS 메일

고객명 *

연락처 / 핸드

테스트 고객

kakao_user_001

주문번호

HSA-240427-01

문의 본문 *

주문한 상품 배송이 언제 도착하는지 알고 싶어요.

백엔드 처리 흐름

웹훅 접수 후 백엔드가 문의를 저장하고 처리합니다.

1 POST /api/v1/webhooks/channels/{channe..

2 { identifier, content }

3 GET /api/admin/inquiries/{id}

4 운영자 검토 · 수정 · 발송

AI 처리 연결

웹훅 접수 후 백엔드가 문의를 저장하고 AI 처리 결과를 상세 화면에서 답변 초안으로 확인합니다. 프론트는 AI 서버를 직접 호출하지 않습니다.

※ AI 서버는 이 화면에서 직접 호출되지 않으며, Backend 웹훅 접수 → 저장 → AI 처리 순서로 동작합니다.

42 / 42